

MODULE 37

PostgreSQL Database Design

Design robust, normalized databases in PostgreSQL and understand how PostgreSQL compares with Oracle in enterprise systems.







MODULE 37 OVERVIEW

Design robust, normalized databases using PostgreSQL and understand how it compares with Oracle in enterprise systems.

Strong database design is the foundation of every reliable enterprise application -- this module covers relational concepts, keys, constraints, normalization, and a hands-on lab designing a real PostgreSQL CRM schema.

DURATION & LAB



1 Hour Theory

Relational concepts, keys, constraints, normalization, and PostgreSQL vs Oracle architecture and enterprise use.



2 Hours Lab

Lab 37: PostgreSQL Design for Customers and Accounts -- a real CRM schema in PostgreSQL.



Scenario

CRM customer/account platform: CUSTOMER, ACCOUNT, ADDRESS, and append-only status history.

MODULE ARC



Understand the Model

Relational concepts, ER modeling, and how PostgreSQL and Oracle both implement them.



Master Keys & Constraints

Primary keys, foreign keys, NOT NULL, UNIQUE, CHECK, referential integrity, normalization.



Build the Lab

Design and implement a real, constrained, normalized PostgreSQL CRM schema end to end.

KEY TAKEAWAY Total time: 3 hours (1 hour theory + 2 hours hands-on lab). By the end, you will design and reason about normalized, constrained databases in both PostgreSQL and Oracle.

WHY RELATIONAL DATABASE DESIGN MATTERS

Learn how to design robust, normalized databases using PostgreSQL and understand how it compares with Oracle in enterprise systems.

WHAT GOOD DESIGN DELIVERS



Design Better Databases

Model entities, attributes, and relationships the right way before writing a single CREATE TABLE.



Ensure Data Integrity

Enforce rules and consistency so the database itself rejects invalid data.



Support Scalable Apps

Well-structured schemas hold up as data and traffic grow.



Make Smarter Decisions

Clean, trustworthy data powers better reporting and analytics.

GOAL & BENEFITS

- **Relational Design** — model data the right way, as tables, rows, and columns with defined relationships.
- **Data Integrity** — enforce rules and constraints so corrupt data cannot enter the system.
- **Performance** — optimize schemas for scale and speed as the application grows.
- **Enterprise Ready** — apply best practices proven in real-world production systems.

GOAL

Build strong data foundations for reliable, secure, and high-performance enterprise applications -- in both PostgreSQL and Oracle.

KEY TAKEAWAY A well-designed relational database is the single foundation every reliable, secure, high-performance enterprise application is built on -- regardless of which database engine runs it.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to design robust, normalized, and secure databases using PostgreSQL while understanding how it compares with Oracle.

- **1. Understand Database Fundamentals** — explain key relational database concepts and the architecture of PostgreSQL and Oracle.
- **2. Work with Database Structures** — create and manage tables, rows, columns, and data types in PostgreSQL.
- **3. Model Data Effectively** — build Entity Relationship (ER) models and identify primary keys, foreign keys, and relationships.
- **4. Apply Constraints and Integrity Rules** — implement NOT NULL, UNIQUE, CHECK constraints and ensure referential integrity.
- **5. Normalize Databases** — apply normalization principles to design efficient, consistent, maintainable databases.
- **6. Design Real-World Databases** — design a normalized customer database using best practices in PostgreSQL.
- **7. Follow Enterprise Best Practices** — apply enterprise database design guidelines for security, performance, and scalability.
- **8. Compare PostgreSQL with Oracle** — identify key similarities and differences to make informed technology decisions.
- **9. Use PostgreSQL Effectively** — use SQL and PostgreSQL tools to create, query, and manage databases.
- **10. Build a Strong Foundation** — develop the skills needed to design reliable, high-performance databases for enterprise applications.

KEY TAKEAWAY Goal: design robust, normalized, and secure databases that meet enterprise requirements using PostgreSQL while understanding how it compares with Oracle.

WHY ENTERPRISES USE POSTGRESQL AND ORACLE (1 OF 2)

Both PostgreSQL and Oracle are powerful, reliable, feature-rich databases that meet the demanding needs of modern enterprise applications.

POSTGRESQL IN ENTERPRISE

- **Open Source and Cost-Effective** — no licensing cost, transparent, and community-driven innovation.
- **High Performance and Reliability** — ACID compliance, MVCC, and a powerful query optimizer ensure reliable performance.
- **Advanced Features** — supports JSON/JSONB, full-text search, window functions, CTEs, partitions, and more.
- **Scalability and Extensibility** — horizontal scaling, replication, and extensible with custom data types, functions, and operators.
- **Wide Adoption and Strong Ecosystem** — used by startups to large enterprises, with strong tooling, partners, and global community support.

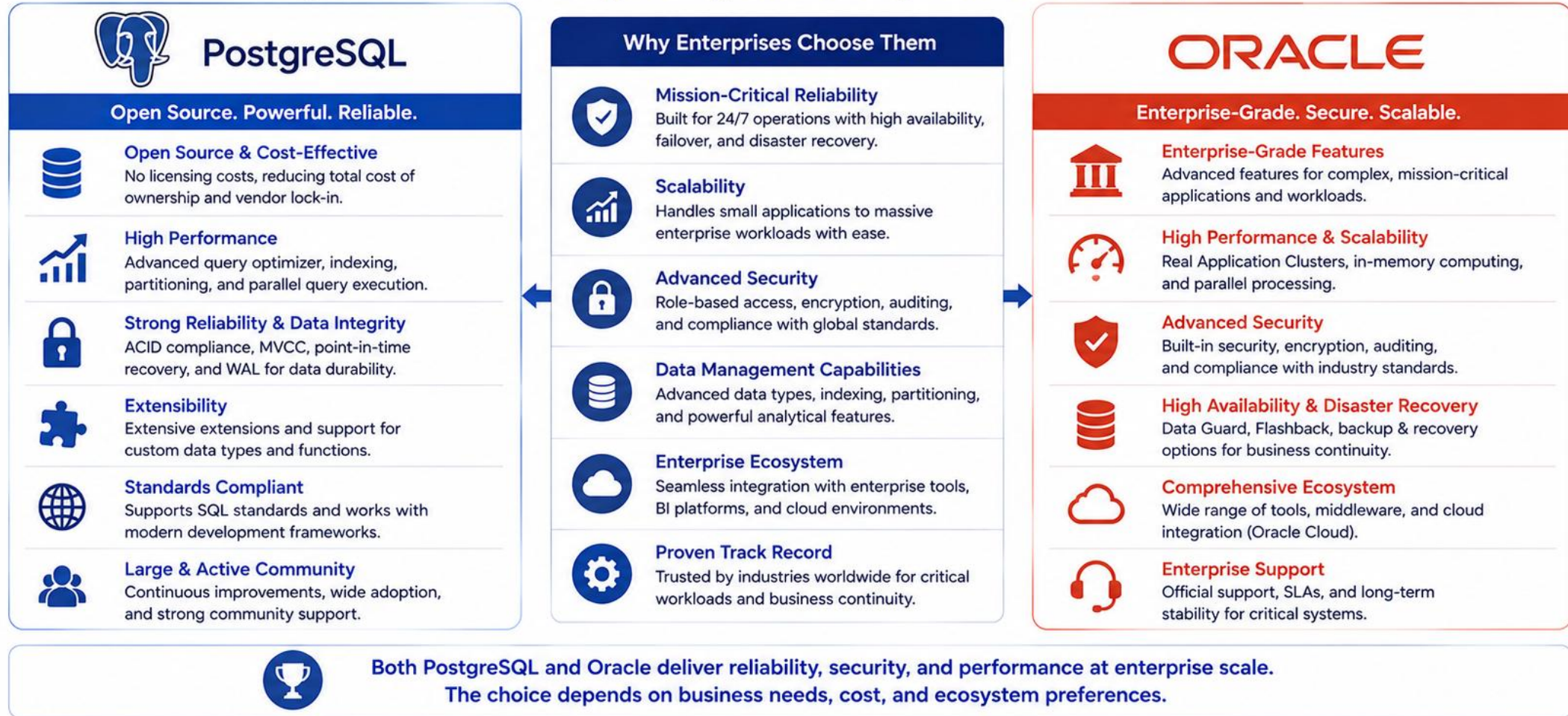
ORACLE IN ENTERPRISE

- **Enterprise Proven and Trusted** — decades of proven reliability for mission-critical, high-availability enterprise workloads.
- **Advanced Security and Compliance** — built-in security, auditing, encryption, and strong compliance capabilities for regulated industries.
- **High Performance and Scalability** — optimized for large-scale deployments with features like Real Application Clusters (RAC) and partitioning.
- **Comprehensive Enterprise Features** — advanced Data Guard, multitenant architecture, in-memory processing, and automated management.
- **Strong Vendor Support** — enterprise-grade support, SLAs, certifications, and a rich professional services ecosystem.

KEY TAKEAWAY PostgreSQL offers flexibility, innovation, and cost efficiency, while Oracle provides enterprise-grade features, security, and proven scalability -- the right choice depends on business needs, budget, and long-term strategy.

Why PostgreSQL and Oracle are Used in Enterprise Systems

High performance, reliability, security, and scalability for mission-critical applications.



WHY ENTERPRISES USE POSTGRESQL AND ORACLE (2 OF 2)

Many organizations use both -- PostgreSQL for agile, cost-effective solutions and Oracle for core, mission-critical systems -- leveraging the strengths of each.

WHAT BOTH DATABASES DELIVER



Data Integrity

Ensure data integrity and consistency.



High Availability & DR

Support high availability and disaster recovery.



Complex Workloads

Handle complex enterprise workloads.



Modern Integration

Integrate with modern applications and the cloud.

IN MODERN ENTERPRISES

Many organizations use both -- PostgreSQL for agile, cost-effective solutions and Oracle for core, mission-critical systems -- leveraging the strengths of each where they deliver the most value.

QUICK SYNTAX COMPARISON

Concept	PostgreSQL	Oracle
Auto-increment	SERIAL / IDENTITY	SEQUENCE + TRIGGER / IDENTITY
Limit rows	LIMIT / OFFSET	ROWNUM / FETCH FIRST
Variable text	VARCHAR	VARCHAR2
Exact decimal	NUMERIC(p,s)	NUMBER(p,s)
Timestamp	TIMESTAMPTZ	TIMESTAMP WITH TIME ZONE

The relational concepts and SQL fundamentals covered today transfer directly between PostgreSQL and Oracle -- syntax differs in details, not in principle.

KEY TAKEAWAY Many organizations use both -- PostgreSQL for agile, cost-effective solutions and Oracle for core, mission-critical systems -- leveraging the strengths of each where they deliver the most value.

RELATIONAL DATABASE CONCEPTS

A relational database organizes data into tables (relations). Data is stored in rows and columns, and relationships exist between tables using keys.

CORE COMPONENTS

- **Table (Relation)** — a collection of related data organized in rows and columns.
- **Row (Tuple)** — a single record in a table.
- **Column (Attribute)** — a field that holds a specific type of data for all rows.
- **Key** — a column or set of columns that uniquely identifies a row.
- **Relationship** — an association between two tables through keys.

CHARACTERISTICS

- **Atomicity** — each value is atomic; it cannot be further divided.
- **Consistency** — rules and constraints ensure valid and consistent data.
- **Referential Integrity** — foreign keys enforce valid relationships between tables.
- **Normalization** — organize data to reduce redundancy and improve integrity.

KEY TAKEAWAY

Structure the data. Define relationships. Enforce rules. Ensure integrity. That's the power of the relational model.

customer_id (PK)	first_name	last_name	email	created_at
1	Aman	Singh	aman@example.com	2024-01-10
2	Priya	Sharma	priya@example.com	2024-01-11

Relational Database Concepts

Organized data in tables with relationships for consistency, integrity, and efficient access.

1. TABLE

Data is stored in tables (relations) of rows and columns.

EmpID	Name	DeptID	Salary
1	Alice	10	60000
2	Bob	20	55000
3	Carol	10	65000

Primary Key: EmpID
Columns: Name, DeptID, Salary

2. PRIMARY KEY

A unique identifier for each row in a table.



EmpID	Name	DeptID
1	Alice	10
2	Bob	20
3	Carol	10

EmpID is the Primary Key

3. FOREIGN KEY

A column (or set of columns) in one table that refers to the Primary Key in another table.

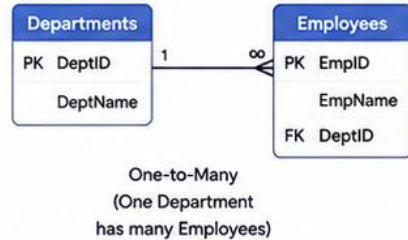
EmpID	Name	DeptID
1	Alice	10
2	Bob	20
3	Carol	10

DeptID	DeptName
10	HR
20	IT
30	Finance

Employees.DeptID is a Foreign Key referencing Departments.DeptID

4. RELATIONSHIP

Defines how tables are related to each other.



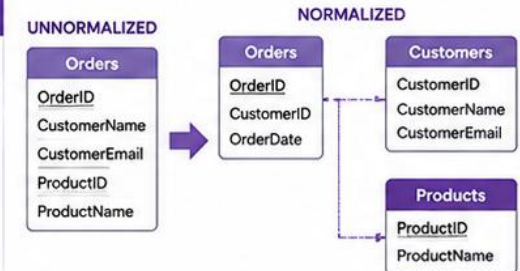
5. CONSTRAINTS

Rules enforced on data to maintain accuracy and reliability.

- ✓ PRIMARY KEY – Ensures each row is unique.
- ✓ FOREIGN KEY – Maintains referential integrity.
- ✓ UNIQUE – Ensures all values are unique.
- ✓ NOT NULL – Prevents null values.
- ✓ CHECK – Ensures values meet a condition.
- ✓ DEFAULT – Sets a default value.

6. NORMALIZATION

Organizing data to minimize redundancy and improve data integrity.



7. SQL OPERATIONS

Perform operations to manage and retrieve data.



- SELECT** – Retrieve data
- INSERT** – Add new data
- UPDATE** – Modify existing data
- DELETE** – Remove data
- JOIN** – Combine data from tables

8. ACID PROPERTIES

Ensure reliable transaction processing.

- A** Atomicity – All or nothing
- C** Consistency – Valid state before and after
- I** Isolation – Transactions don't interfere
- D** Durability – Changes are permanent

9. INDEXING

Speeds up data retrieval operations.



- ✓ Improves SELECT performance
- ✓ Speeds up WHERE, JOIN, ORDER BY
- ✓ Uses extra storage
- ✓ Choose indexes wisely

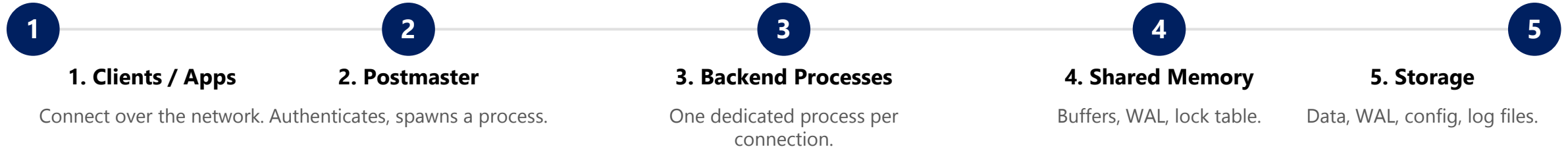


KEY TAKEAWAY

Relational database concepts help design structured, consistent, and scalable databases that power modern enterprise applications.

POSTGRESQL AND ORACLE DATABASE ARCHITECTURE (1 OF 2)

Both databases follow a client-server model but use different internal architectures. Here is how PostgreSQL is built.



HOW POSTGRESQL IS BUILT

- **Process-Per-Connection Model** — the Postmaster authenticates each client connection and allocates a dedicated backend server process for it.
- **Shared Memory** — shared buffers (data cache), WAL buffers (write-ahead log), and a lock table are shared across all backend processes.
- **Storage** — data files, WAL files, configuration files, and log files on disk.
- **Extensible and Open-Source** — the architecture is designed to be simple, reliable, and easy to extend.

POSTGRESQL HIGHLIGHTS

Simple & Reliable

Strong Consistency (MVCC)

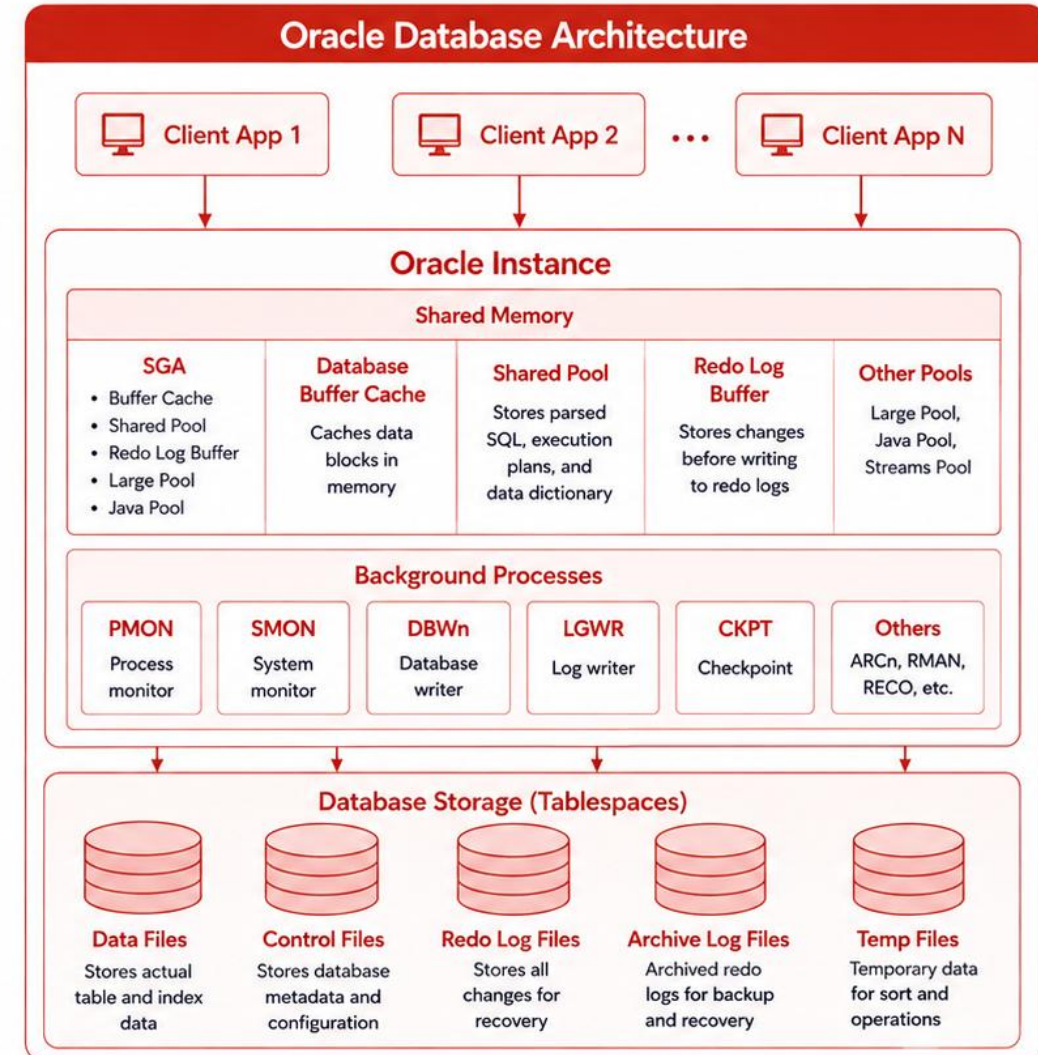
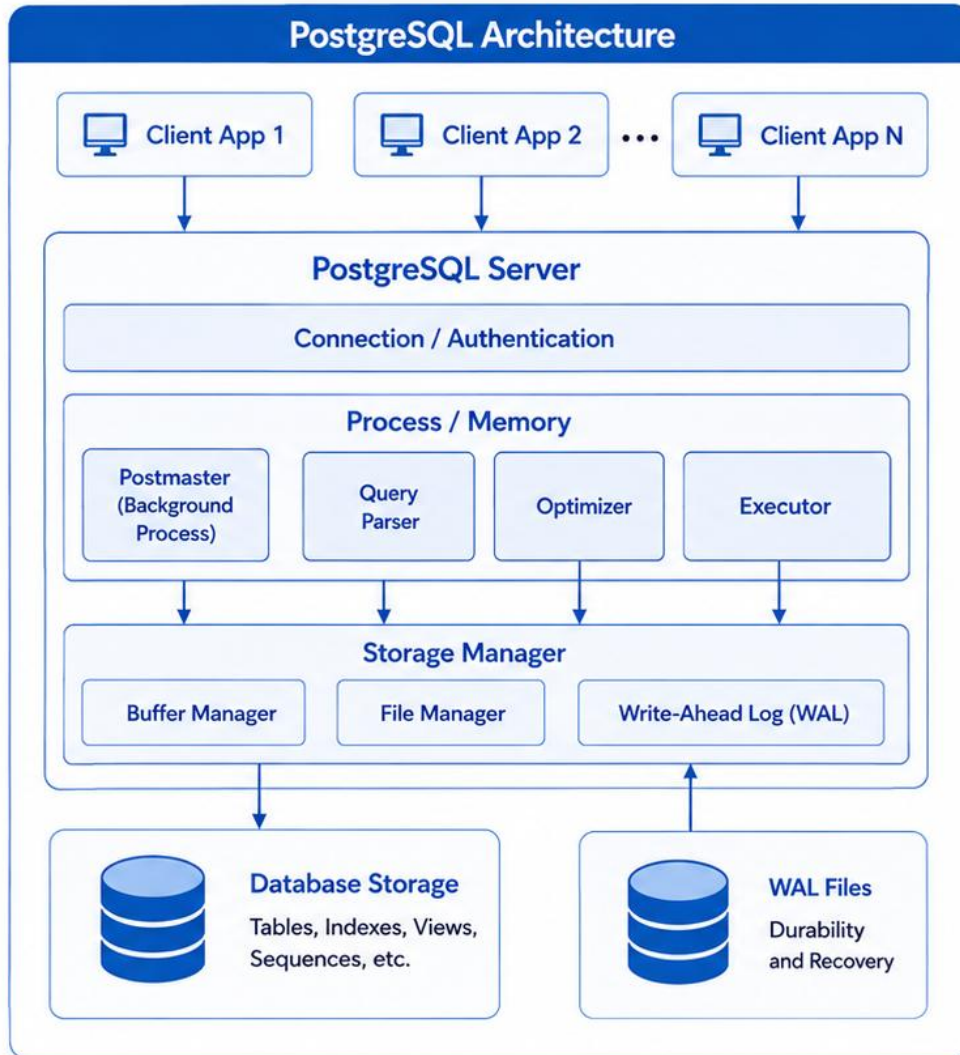
Extensible

Cost-Effective

Each connection is handled by its own dedicated backend process, not a thread -- this gives PostgreSQL strong isolation between sessions, at the cost of somewhat higher per-connection memory overhead. Custom data types, functions, and operators make the engine highly extensible.

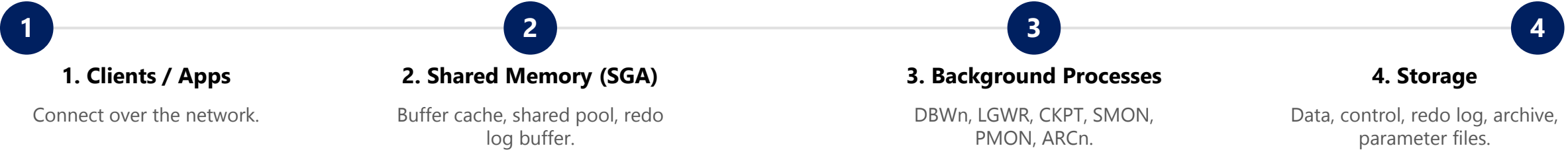
KEY TAKEAWAY PostgreSQL uses a process-per-connection model with a shared storage engine: the Postmaster spawns a dedicated backend process per client, and all processes share buffers, WAL, and a lock table.

PostgreSQL and Oracle Database Architecture



POSTGRESQL AND ORACLE DATABASE ARCHITECTURE (2 OF 2)

Oracle uses a multi-process architecture with shared memory (the SGA) and background processes for high scalability.



HOW ORACLE IS BUILT

- **Shared Memory (SGA)** — the System Global Area holds the Database Buffer Cache, Shared Pool, Redo Log Buffer, Large Pool, and Java Pool.
- **Background Processes** — DBWn (Database Writer), LGWR (Log Writer), CKPT (Checkpoint), SMON (System Monitor), PMON (Process Monitor), ARCn (Archiver), and others.
- **Storage** — data files, control files, redo log files, archive logs, and parameter files.
- **Multi-Process, Built for Scale** — background processes handle critical database tasks in parallel, tuned for high availability at large scale.

SIDE-BY-SIDE: KEY TAKEAWAY

Aspect	PostgreSQL	Oracle
Connection model	Process-per-connection	Multi-process + SGA
Shared memory	Shared buffers, WAL, locks	SGA: buffer cache, pools
Background tasks	Minimal, lightweight	DBWn, LGWR, CKPT, SMON...
Best fit	Extensible, cost-effective	High-availability, large scale

KEY TAKEAWAY PostgreSQL uses a process-per-connection model with a shared storage engine; Oracle uses a multi-process architecture with shared memory (the SGA) and background processes -- both are built for reliability, just differently.

TABLES, ROWS AND COLUMNS

Data in a relational database is organized in tables. Each table is made up of rows and columns.

THE THREE BUILDING BLOCKS

- **Table** — stores data about one entity or business object.
- **Row (Tuple)** — a single record or instance of the entity.
- **Column (Attribute)** — a specific attribute or property of the entity.

EXAMPLE: CUSTOMER TABLE

customer_id (PK)	first_name	last_name	email	phone
1	Aman	Singh	aman@example.com	555-0101
2	Priya	Sharma	priya@example.com	555-0102
3	Rahul	Verma	rahul@example.com	555-0103

KEY TAKEAWAY Tables organize related data, columns define the attributes, rows store the actual data, and a primary key uniquely identifies each row -- understanding this is the foundation for designing, querying, and managing relational databases in both PostgreSQL and Oracle.

Tables, Rows and Columns

Organizing data in a table structure



TABLE

A table is a collection of related data items organized in rows and columns.

ROWS

COLUMNS

EmpID	FirstName	LastName	Email	Department	Salary	HireDate
101	Alice	Johnson	alice.johnson@company.com	HR	65000	2022-01-15
102	Bob	Smith	bob.smith@company.com	IT	75000	2021-03-22
103	Carol	Williams	carol.williams@company.com	Finance	70000	2020-07-10
104	David	Brown	david.brown@company.com	Marketing	68000	2019-11-05
105	Eve	Davis	eve.davis@company.com	IT	72000	2023-02-18

A ROW (Record)

Represents a single record in the table.

A COLUMN (Field)

Represents a specific attribute of the data.



KEY POINTS

- Columns define the structure.
- Rows hold the actual data.
- Each row is a record.
- Each column is a field (attribute).



EXAMPLE

In the Employees table:

- Each row is an employee.
- Each column stores a specific information about the employee.

ENTITY RELATIONSHIP MODELING (ER MODELING)

ER Modeling is a conceptual approach to designing databases by defining entities, their attributes, and the relationships between them.

CORE COMPONENTS

- **Entity** — a real-world object or concept about which data is stored.
- **Attribute** — a property or characteristic of an entity.
- **Key Attribute** — attribute(s) that uniquely identify an entity.
- **Relationship** — an association between two or more entities.

EXAMPLE: ONLINE BANKING SYSTEM (ER)

Entity	Key Attribute	Relationship	Cardinality
CUSTOMER	customer_id	CUSTOMER owns ACCOUNT	1 : M
ACCOUNT	account_id	ACCOUNT has TRANSACTION	1 : M
TRANSACTION	transaction_id	belongs to one ACCOUNT	M : 1

CARDINALITY NOTATION

1 = One

M = Many (One or more)

1:1 One-to-One

1:M One-to-Many

M:M Many-to-Many

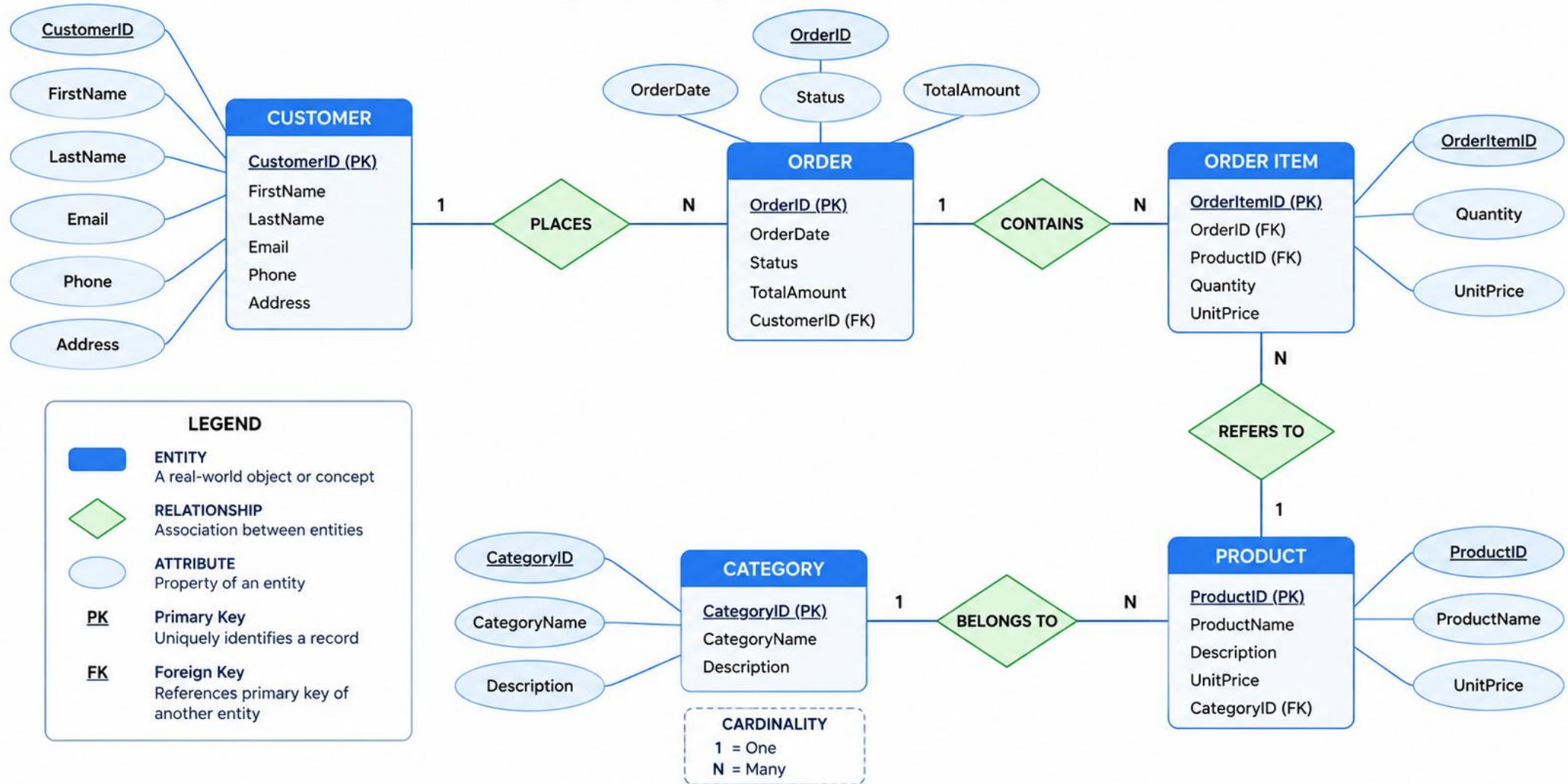
BENEFITS OF ER MODELING

- Provides a clear understanding of data and business rules.
- Helps design efficient, well-structured databases.
- Improves communication between business and technical teams.
- Reduces data redundancy and serves as a blueprint for implementation.

KEY TAKEAWAY ER modeling is the first and most important step in database design -- it captures real-world entities, attributes, and relationships before a single table is created, in either PostgreSQL or Oracle.

Entity Relationship Modeling

Designing the logical structure of a database



TRY IT YOURSELF — EXERCISE 1: CRM ENTITIES

Reinforces: identifying persistent entities and attributes for Northstar CRM's customer/account design -- an analysis exercise.

Candidate Entities & Attributes (from exercise-01-entities.md)

```
CUSTOMER -- customer_id, full_name, status, created_at
ACCOUNT  -- account_id, customer_id, account_number, type
ADDRESS  -- add only if this exercise's scenario genuinely needs it
```

STEPS (RECORD IN notes/lab37-design.md)

Step	What To Do
1 -- Entities	Propose customer and account as the two core tables; add address only if the scenario genuinely needs it.
2 -- Attributes	List CUSTOMER: customer_id, full_name, status, created_at. List ACCOUNT: account_id, customer_id, account_number, type.
3 -- Fixtures	Plan the seed rows you will reuse all module: Amina Khan as CUS-1001 and Ravi Singh as CUS-1002.
4 -- Notes	Save your entity and attribute list to notes/lab37-design.md before moving to Exercise 2.

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 1 before continuing: exercises/exercise-01-entities.md (workspace: examples/module-37-exercises).

PRIMARY KEYS

A Primary Key is a column (or set of columns) that uniquely identifies each row in a table.

RULES FOR A PRIMARY KEY

- **Must Be Unique** — no two rows can have the same value.
- **Cannot Be NULL** — every row must have a value.
- **Only ONE Per Table** — a table can have only one primary key.
- **Single or Composite** — it can be one column or a combination of columns.

BENEFITS

- Ensures each row is unique and prevents duplicate data.
- Improves data integrity and reliability.
- Used by other tables as a foreign key reference.

SQL Example (PostgreSQL)

```
CREATE TABLE customer (  
    customer_id SERIAL PRIMARY KEY,  
    first_name  VARCHAR(50),  
    last_name   VARCHAR(50),  
    email       VARCHAR(100)  
);
```

BEST PRACTICE

Choose a primary key that is simple, stable, and never changes -- a surrogate ID, not a natural value like email, which can change over time.

KEY TAKEAWAY A primary key uniquely identifies every row, can never be NULL, and there is only ever one per table -- it is the anchor every foreign key relationship depends on.

Primary Keys

A Primary Key uniquely identifies each record in a table.

EXAMPLE TABLE: STUDENT

StudentID (Primary Key)	FirstName	LastName	Email	DateOfBirth
101	Ava	Johnson	ava.johnson@email.com	2001-03-15
102	Liam	Smith	liam.smith@email.com	2000-07-22
103	Emma	Williams	emma.williams@email.com	2002-11-30
104	Noah	Brown	noah.brown@email.com	2001-09-05
105	Olivia	Davis	olivia.davis@email.com	2000-12-18



Primary Key

The StudentID uniquely identifies each student. No two rows can have the same value in this column.



No Duplicate Values

Primary Key values must be unique and cannot be NULL.

KEY CHARACTERISTICS



Unique

No two records can have the same primary key value.



Not NULL

Primary key columns cannot have NULL values.



Stable

Primary key values should not change over time.



Identifies Records

Used to uniquely identify and access each record.

EXAMPLE SQL

```
CREATE TABLE Student (  
  StudentID INT PRIMARY KEY,  
  FirstName VARCHAR(50),  
  LastName VARCHAR(50),  
  Email VARCHAR(100),  
  DateOfBirth DATE  
);
```

FOREIGN KEYS

A Foreign Key is a column (or set of columns) in one table that refers to the Primary Key of another table, enforcing referential integrity.

PARENT: CUSTOMER • CHILD: ORDER

customer_id (PK)	first_name	email
101	Aman	aman@example.com
102	Priya	priya@example.com

order_id (PK)	customer_id (FK)	order_date	amount
5001	101	2024-05-01	250.00
5002	102	2024-05-02	120.00
5003	101	2024-05-03	75.50

Every customer_id value in ORDER must already exist in CUSTOMER.customer_id.

KEY POINTS

- References the primary key of another table.
- Enforces referential integrity; prevents orphan records.
- Can have duplicate values and can be NULL (unless NOT NULL).

SQL Example (PostgreSQL)

```
CREATE TABLE orders (  
  order_id SERIAL PRIMARY KEY,  
  customer_id INT NOT NULL,  
  order_date DATE,  
  amount NUMERIC(10,2),  
  CONSTRAINT fk_order_customer  
    FOREIGN KEY (customer_id)  
    REFERENCES customer(customer_id)  
);
```

EXAMPLE SCENARIO

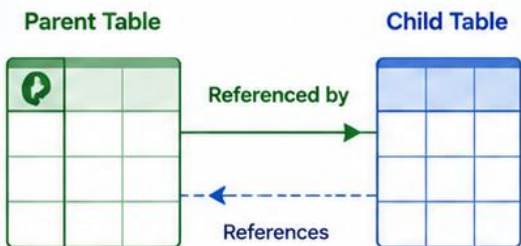
- Valid: insert order with customer_id = 101 (exists) → success.
- Invalid: insert order with customer_id = 999 (does not exist) → error.

KEY TAKEAWAY Foreign keys create links between tables, ensuring related data stays accurate, consistent, and trustworthy -- an order can never point at a customer that doesn't exist.

Foreign Keys

A foreign key is a column (or set of columns) in one table that refers to the primary key in another table.

How It Works



The child table stores a value that matches the primary key value in the parent table.

Key Points

-  **Maintains Integrity**
Ensures data consistency between tables.
-  **Enforces Relationships**
Prevents invalid or orphan records in the child table.
-  **Defines Dependency**
Child table depends on the parent table for valid data.

Example: Orders and Customers

Customers (Parent Table)

CustomerID (PK)	Name	Email
1	Alice Johnson	alice@example.com
2	Bob Smith	bob@example.com
3	Carol Williams	carol@example.com

Orders (Child Table)

OrderID (PK)	CustomerID (FK)	OrderDate	Amount
101	1	2024-05-01	250.00
102	2	2024-05-02	150.00
103	1	2024-05-03	300.00
104	3	2024-05-04	200.00

SQL Example

```
CREATE TABLE Customers (  
    CustomerID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Email VARCHAR(100)  
);  
  
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    OrderDate DATE,  
    Amount DECIMAL(10,2),  
    FOREIGN KEY (CustomerID)  
        REFERENCES Customers(CustomerID)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT  
);
```

Benefits

-  **Data Consistency**
Keeps related data accurate and reliable.
-  **Referential Integrity**
Ensures relationships remain valid.
-  **Prevents Orphan Records**
Stops records that reference non-existent data.

RELATIONSHIPS IN DATABASES

A relationship defines how two or more tables (entities) are associated with each other -- a fundamental part of relational database design.

Type	Notation	Example	How It's Implemented
One-to-One	1 : 1	Customer ↔ Customer Profile	FK with a UNIQUE constraint on the child.
One-to-Many	1 : M	Customer → Orders	FK on the 'many' side referencing the 'one' side.
Many-to-One	M : 1	Orders → Customer	Same FK, read from the other direction.
Many-to-Many	M : M	Orders ↔ Products	Resolved with a junction table (e.g. ORDER_ITEM).

JUNCTION TABLE: ORDER_ITEM

order_id (PK,FK)	product_id (PK,FK)	quantity	price
5001	P201	1	800.00

ORDER_ITEM resolves the many-to-many relationship: each row links one order to one product.

CARDINALITY NOTATION

1 One (Exactly one)

M Many (One or more)

0 Zero or one (Optional)

* Many (Crow's foot)

Real-world analogy: a customer places many orders; an order contains many products; a product can be part of many orders -- each order item links exactly one order with exactly one product.

KEY TAKEAWAY Proper relationships reduce data redundancy and improve integrity. Use foreign keys to enforce relationships; resolve many-to-many relationships with a junction table.

Relationships in Databases

Relationships define how tables (entities) are connected using keys.

1. One-to-One (1:1)

Each record in Table A is related to only one record in Table B, and vice versa.

Users			UserProfiles	
UserID	Name		UserID (FK)	Email
1	Alice	1	1	alice@example.com
2	Bob	2	2	bob@example.com
3	Carol	3	3	carol@example.com

Example: Users and UserProfiles



- UserProfiles.UserID is both a Foreign Key and a Unique Key.
- Each user has one profile, each profile belongs to one user.

2. One-to-Many (1:N)

One record in Table A can be related to multiple records in Table B. Each record in Table B is related to one record in Table A.

Customers			Orders	
CustomerID	Name		OrderID	CustomerID (FK)
1	Alice	1	101	1
2	Bob	2	102	1
3	Carol	3	103	2
			104	3
			105	3

Example: Customers and Orders



- Orders.CustomerID is a Foreign Key.
- One customer can have many orders, but each order belongs to one customer.

3. Many-to-Many (M:N)

Multiple records in Table A can be related to multiple records in Table B. Implemented using a junction table.

Students			Courses	
StudentID	Name		CourseID	Title
1	Alice	1	10	Math
2	Bob	2	20	Science
3	Carol	3	30	History

Enrollments	
StudentID (FK)	CourseID (FK)
1	10
1	20
2	10
3	20
3	30

Example: Students and Courses via Enrollments



- Enrollments is a junction table.
- StudentID and CourseID are Foreign Keys.

4. Self-Referencing (Recursive)

A table is related to itself using a Foreign Key.

Employees		
EmployeeID	Name	ManagerID (FK)
1	CEO	NULL
2	Alice	1
3	Bob	1
4	Carol	2
5	Dave	2

Example: Employee reporting structure



- Employees.ManagerID is a Foreign Key referencing Employees.EmployeeID.
- An employee can have one manager, a manager can have many employees.

Key Concepts



Primary Key (PK)
Uniquely identifies each record in a table.



Foreign Key (FK)
References the Primary Key in another table.



Referential Integrity
Ensures valid relationships between related tables are maintained.



Uses
Model real-world data, reduce redundancy, improve data integrity.

TRY IT YOURSELF — EXERCISE 2: ER SKETCH

Reinforces: customer-account cardinality and an ON DELETE policy decision -- an architecture exercise.

REFERENCE: CARDINALITY & KEYS

Rule	Detail
customer → account	1 : N cardinality
account.customer_id	FK → customer.customer_id
customer.customer_id	PK / unique business key

ER Sketch (Mermaid or ASCII)

```
erDiagram
    CUSTOMER ||--o{ ACCOUNT : owns
    CUSTOMER {
        string customer_id PK
    }
    ACCOUNT {
        string account_id PK
        string customer_id FK
    }
```

STEPS (RECORD IN notes/lab37-design.md)

Step	What To Do
1 -- Copy Rules	Copy the cardinality and key reference table (left) into your own notes exactly as shown.
2 -- Diagram	Sketch Customer --o{ Account in Mermaid or plain ASCII, matching the example at left.
3 -- Cascade Policy	Decide ON DELETE RESTRICT vs CASCADE for account.customer_id, and justify your choice in one sentence.
4 -- Boundary	Do not create Kafka outbox tables in this exercise -- unless the guide explicitly asks for one.

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 2 before continuing: exercises/exercise-02-er-sketch.md (workspace: examples/module-37-exercises).

NOT NULL CONSTRAINTS

A NOT NULL constraint ensures that a column cannot have a NULL value -- it enforces data completeness and improves data integrity.

EXAMPLE: CUSTOMER TABLE

name (NOT NULL)	email (NOT NULL)	phone (NOT NULL)	date_of_birth (NULL OK)
Aman Singh	aman@example.com	555-0101	1990-05-10
Priya Sharma	priya@example.com	555-0102	NULL

SYNTAX (POSTGRESQL VS ORACLE)

```
column_name data_type NOT NULL
-- identical syntax in PostgreSQL and Oracle
```

EXAMPLES OF USE

- **Should be NOT NULL** — primary keys, name, email, phone, order date, total amount.
- **Can allow NULL** — middle name, date of birth, optional address fields.

INSERT: valid vs invalid

```
INSERT INTO customer
(customer_id, name, email, phone)
VALUES (104, 'Neha Patel',
       'neha@example.com', '555-0104');
-- Insert successful. (valid)

INSERT INTO customer
(customer_id, name, email, phone)
VALUES (105, NULL, 'x@example.com', NULL);
-- ERROR: null value in column "name"
-- violates not-null constraint
```

BEST PRACTICE

Always define NOT NULL for mandatory data at the database level -- don't rely only on application code.

KEY TAKEAWAY NOT NULL means a value is required; NULL (allowed) means the value is optional and can be missing. Choose based on business rules and data requirements.

NOT NULL Constraints

Ensures that a column cannot have NULL values. A value must be provided for the column.

What is NOT NULL?



The **NOT NULL** constraint enforces that a column always contains a value. It prevents **NULL** (unknown or missing) values from being stored.

Benefits

- ✓ Improves data integrity
- ✓ Ensures required data is always provided
- ✓ Prevents incomplete or invalid records
- ✓ Simplifies application logic
- ✓ Helps in enforcing business rules

Syntax

```
CREATE TABLE table_name (  
    column_name datatype NOT NULL,  
    ...  
);  
  
ALTER TABLE table_name  
ALTER COLUMN column_name datatype NOT NULL;
```

Key Points

- **NOT NULL** is a column-level constraint.
- It can be set at the time of table creation or added later.
- It does not apply a default value.
- If a column is **NOT NULL**, every row must have a value for that column.

Example: Employees Table

Table Structure

Column Name	Data Type	Constraints	Description
EmployeeID	INT	PRIMARY KEY, NOT NULL	Unique ID for employee
Name	VARCHAR(100)	NOT NULL	Employee name
Email	VARCHAR(100)	NOT NULL	Employee email address
Department	VARCHAR(50)	NULL	Employee department
Salary	DECIMAL(10,2)	NOT NULL	Employee salary

Create Table

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY NOT NULL,  
    Name VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) NOT NULL,  
    Department VARCHAR(50),  
    Salary DECIMAL(10,2) NOT NULL  
);
```

Add NOT NULL to Existing Column

```
ALTER TABLE Employees  
ALTER COLUMN Department VARCHAR(50) NOT NULL;
```

Inserting Data

✓ Valid Insert (All NOT NULL columns have values)

```
INSERT INTO Employees (EmployeeID, Name, Email, Department, Salary)  
VALUES (1, 'Alice Johnson', 'alice@example.com', 'HR', 60000.00);
```

✗ Invalid Insert (Missing value for NOT NULL column)

```
INSERT INTO Employees (EmployeeID, Name, Department, Salary)  
VALUES (2, NULL, 'IT', 55000.00);  
-- Error: NULL value in column 'Name' violates NOT NULL constraint.
```

Resulting Data

EmployeeID	Name	Email	Department	Salary
1	Alice Johnson	alice@example.com	HR	60000.00

UNIQUE CONSTRAINTS

A UNIQUE constraint ensures that all values in a column (or set of columns) are distinct across the table.

RULES OF UNIQUE

- **All Values Must Be Unique** — no two rows can have the same value.
- **Allows One NULL** — in most databases, including PostgreSQL and Oracle.
- **Column or Column Set** — applied at the column level, or across multiple columns.
- **Not the Same as PRIMARY KEY** — a table can have multiple UNIQUE constraints, but only one primary key.

EXAMPLES OF USE

Email (one per user)

Username

Phone Number

Employee ID

Syntax (PostgreSQL vs Oracle)

```
-- PostgreSQL
email VARCHAR(100) UNIQUE,
CONSTRAINT uk_customer_email
    UNIQUE (email)

-- Oracle: identical syntax
email VARCHAR2(100) UNIQUE
```

WHAT HAPPENS ON A DUPLICATE?

Insert email = 'priya@example.com' a second time: ERROR -- duplicate key value violates unique constraint "uk_customer_email". Detail: Key (email)=(priya@example.com) already exists.

KEY TAKEAWAY UNIQUE ensures distinct values, while NOT NULL ensures presence of a value -- use UNIQUE on columns like email, phone, or username to keep data clean, accurate, and reliable.

UNIQUE Constraints

Ensures that all values in a column (or set of columns) are different across rows.

Key Points

- Enforces uniqueness of values.
- Allows NULL values (multiple NULLs are allowed).
- Applies to a single column or multiple columns (composite UNIQUE).
- Helps maintain data integrity.

Syntax

Column-level UNIQUE

```
CREATE TABLE table_name (  
  column_name data_type UNIQUE,  
  ...  
);
```

Table-level UNIQUE

```
CREATE TABLE table_name (  
  column1 data_type,  
  column2 data_type,  
  UNIQUE (column1, column2)  
);
```

Example Table

Users

id (PK)	username (UNIQUE)	email (UNIQUE)	phone (UNIQUE)
1	alice	alice@example.com	555-0101
2	bob	bob@example.com	555-0102
3	charlie	charlie@example.com	555-0103
4	diana	diana@example.com	555-0104

✓ Allowed (Unique)

id	username (UNIQUE)	email (UNIQUE)	phone (UNIQUE)
5	eve	eve@example.com	555-0105
6	frank	frank@example.com	555-0106

✗ Blocked (Violates UNIQUE)

id	username (UNIQUE)	email (UNIQUE)	phone (UNIQUE)
7	alice	grace@example.com	555-0107
8	henry	bob@example.com	555-0108
9	irene	irene@example.com	555-0102

→ Duplicate username
→ Duplicate email
→ Duplicate phone

CHECK CONSTRAINTS

A CHECK constraint restricts the values that can be stored in a column by enforcing a specific condition or rule.

EXAMPLES OF CHECK CONDITIONS

- age >= 18
- salary > 0
- status IN ('PROSPECT', 'ACTIVE', 'SUSPENDED', 'CLOSED')
- start_date <= end_date
- discount_percent BETWEEN 0 AND 100

RULES OF CHECK

- The condition must evaluate to TRUE for the data to be accepted.
- If the condition is FALSE, the insert or update is rejected.
- Can be defined at column level or table level.

CREATE TABLE (PostgreSQL)

```
CREATE TABLE customer (  
  customer_id SERIAL PRIMARY KEY,  
  status VARCHAR(20) DEFAULT 'PROSPECT',  
  CONSTRAINT ck_customer_status  
    CHECK (status IN (  
      'PROSPECT', 'ACTIVE',  
      'SUSPENDED', 'CLOSED'  
    ))  
);
```

INVALID INSERT EXAMPLE

INSERT status = 'UNKNOWN' -> ERROR: new row violates check constraint "ck_customer_status".

KEY TAKEAWAY CHECK constraints enforce business rules and valid data ranges directly in the database, ensuring only correct and meaningful data is stored.

CHECK Constraints

Ensures that all values in a column (or set of columns) satisfy a specified condition.

Key Points

- Enforces domain-specific business rules.
- Condition is evaluated for each row at INSERT or UPDATE.
- If the condition is false, the operation is rejected.
- Helps maintain accurate and valid data.

Syntax

Column-level CHECK

```
CREATE TABLE table_name (  
  column_name data_type CHECK (condition),  
  ...  
);
```

Table-level CHECK

```
CREATE TABLE table_name (  
  column1 data_type,  
  column2 data_type,  
  ...  
  CHECK (condition)  
);
```

Example

Rule: **age** must be between 18 and 60, and **salary** must be greater than 0.

```
CREATE TABLE Employees (  
  id INT PRIMARY KEY,  
  name VARCHAR(50) NOT NULL,  
  age INT CHECK (age BETWEEN 18 AND 60),  
  salary DECIMAL(10,2) CHECK (salary > 0)  
);
```

✓ Allowed (Satisfies CHECK)

id	name	age (18–60)	salary (> 0)
1	Alice	25	50000.00
2	Bob	40	75000.50

✗ Rejected (Violates CHECK)

id	name	age (18–60)	salary (> 0)	Reason
3	Charlie	17	50000.00	Age < 18
4	David	65	60000.00	Age > 60
5	Eve	30	0	Salary not > 0
6	Frank	30	-1000.00	Salary not > 0

TRY IT YOURSELF — EXERCISE 3: CONSTRAINTS CHECKLIST

Reinforces: choosing PK, UNIQUE, CHECK, and NOT NULL constraints before writing DDL -- a documentation exercise.

CONSTRAINTS CHECKLIST (RECORD IN `notes/lab37-design.md`)

Step	What To Do
1 -- PK / UK	Put a primary key on <code>customer_id</code> , and a UNIQUE constraint on <code>account_number</code> .
2 -- CHECK	Draft a CHECK for <code>status</code> , e.g. <code>status IN ('ACTIVE','SUSPENDED', ...)</code> -- list every value you intend to allow.
3 -- NOT NULL	Mark <code>full_name</code> and <code>status</code> as NOT NULL; decide which other columns are truly mandatory.
4 -- SQLSTATE Awareness	Note that a UNIQUE violation raises SQLSTATE 23505 -- you'll trigger this on purpose in Lab 37's negative tests.

Sample Constraint Notes (fill in your own before Ex. 4)

```
-- customer_id
  PK
-- account_number
  UNIQUE
-- full_name
  NOT NULL
-- status
  NOT NULL,
  CHECK (status IN
    ('ACTIVE', 'SUSPENDED',
     ...))
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 3 before continuing: `exercises/exercise-03-constraints.md` (workspace: `examples/module-37-exercises`).

REFERENTIAL INTEGRITY

Referential Integrity ensures that relationships between related tables remain consistent and valid -- a foreign key value must match an existing primary key value.

HOW IT WORKS

- When a row is inserted or updated in the child table, the database checks if the foreign key value exists in the parent table.
- If it exists, the operation succeeds. If it does not exist, the operation fails.

VALID VS INVALID INSERT

- Valid: order with customer_id = 102 -> 102 exists in CUSTOMER -> success.
- Invalid: order with customer_id = 999 -> 999 not in CUSTOMER -> rejected.

ENFORCEMENT ACTIONS (PARENT ROW MODIFIED)

Action	Description
RESTRICT	Prevents delete/update if child rows exist (default in PostgreSQL, Oracle).
CASCADE	Automatically deletes/updates child rows when the parent row changes.
SET NULL	Sets the foreign key in child rows to NULL when the parent row changes.
SET DEFAULT	Sets the foreign key to its default value when the parent row changes.

KEY CONCEPT

Parent Table (Primary Key: unique and must exist) -> Child Table (Foreign Key: must reference an existing parent key). Referential integrity is automatically enforced by the database once foreign keys are defined.

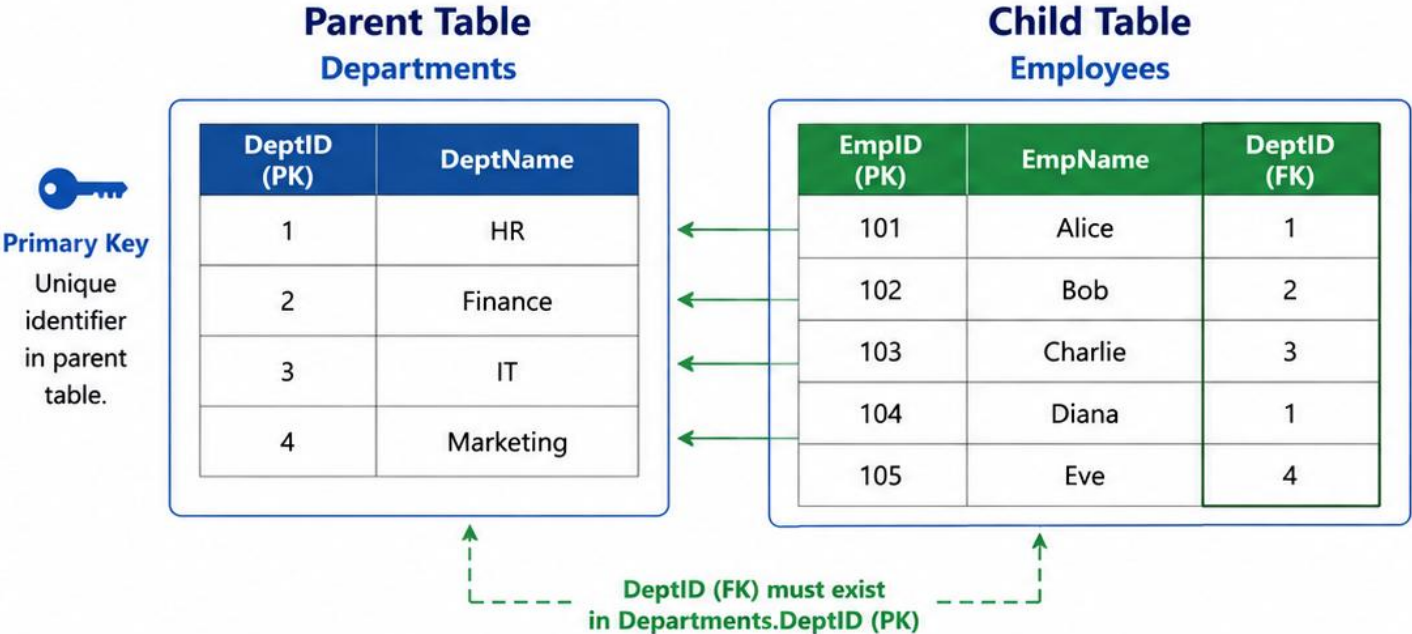
Both PostgreSQL and Oracle support referential integrity through FOREIGN KEY constraints -- the rule, and its enforcement, is identical in principle.

KEY TAKEAWAY Referential integrity ensures that every child record has a valid parent -- it protects relationships and keeps your database reliable and consistent.

Referential Integrity

Ensures consistency of relationships between tables.

A value in a foreign key column must match an existing primary key value in the referenced table.



How It Works

You can insert a row in the child table only if the foreign key value exists in the parent table.

If a referenced parent row is updated or deleted, the action is restricted or handled according to the defined rule.

Referential Actions (on Delete/Update)



RESTRICT

Prevent the operation if child rows exist.



CASCADE

Automatically delete/update child rows.



SET NULL

Set the foreign key to NULL in child rows.



NO ACTION

Similar to RESTRICT (checked at end of transaction).



Valid Operations

- Insert employee with DeptID = 2 ✓ (exists in Departments)
- Update employee DeptID from 3 to 4 ✓ (4 exists in Departments)
- Delete department with DeptID = 4 only if no employee references it (based on rule)



Invalid Operations

- Insert employee with DeptID = 10 ✗ (10 does not exist)
- Update employee DeptID to 99 ✗ (99 does not exist)
- Delete department with DeptID = 2 if employees still reference it (based on rule)

NORMALIZATION FUNDAMENTALS (1 OF 2)

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity.

GOALS OF NORMALIZATION

- Reduce data redundancy.
- Eliminate update, insert, and delete anomalies.
- Ensure data dependencies.
- Improve data integrity.

EXAMPLE: UNNORMALIZED TABLE

OrderID	CustomerID	CustomerName	ProductName	Quantity
1001	C101	Aman Singh	Laptop	1
1001	C101	Aman Singh	Mouse	2
1002	C102	Priya Sharma	Keyboard	1

Problem: customer and product details repeat across rows, leading directly to update, insert, and delete anomalies.

TYPES OF ANOMALIES

- **Update Anomaly** — the same data stored in multiple rows; updating one copy risks inconsistency.
- **Insert Anomaly** — cannot insert data without unrelated data also being present.
- **Delete Anomaly** — deleting a row may cause loss of unrelated, still-needed data.

BENEFITS OF NORMALIZATION

**Less Redundancy**
Store data once, save storage.

**Better Integrity**
Accurate, consistent data.

**Easier Upkeep**
Updates and changes are simpler.

**Scalability**
Supports growth and performance.

KEY TAKEAWAY Normalization is a systematic approach to decomposing large tables into smaller, well-structured tables with defined relationships -- eliminating update, insert, and delete anomalies.

Normalization Fundamentals

Organizing data to reduce redundancy and improve data integrity.



Goals of Normalization

- Minimize data redundancy
- Eliminate update anomalies
- Ensure data integrity
- Improve maintainability



Insertion Anomaly

Cannot insert data without another unrelated data.



Update Anomaly

Same data stored in multiple places leads to inconsistency.



Deletion Anomaly

Deleting a row may cause loss of other useful data.

UNNORMALIZED (0NF)

All data in a single table

Orders_Unnormalized					
OrderID	CustomerID	CustomerName	ProductID	ProductName	Quantity
1001	C01	Alice	P01	Laptop	1
1001	C01	Alice	P02	Mouse	2
1002	C02	Bob	P01	Laptop	1
1002	C02	Bob	P03	Keyboard	1

Issues

- Redundant data
- Update, insertion and deletion anomalies

1NF

Eliminate repeating groups

Orders_1NF					
OrderID	CustomerID	CustomerName	ProductID	ProductName	Quantity
1001	C01	Alice	P01	Laptop	1
1001	C01	Alice	P02	Mouse	2
1002	C02	Bob	P01	Laptop	1
1002	C02	Bob	P03	Keyboard	1

Result

- Each column has atomic values
- No repeating groups

2NF

Remove partial dependencies

Orders		
OrderID	CustomerID	CustomerName
1001	C01	Alice
1002	C02	Bob

OrderItems			
OrderID	ProductID	ProductName	Quantity
1001	P01	Laptop	1
1001	P02	Mouse	2
1002	P01	Laptop	1
1002	P03	Keyboard	1

Result

- Partial dependency removed
- Non-key attributes depend on the whole primary key

3NF

Remove transitive dependencies

Orders	
OrderID	CustomerID
1001	C01
1002	C02

Customers	
CustomerID	CustomerName
C01	Alice
C02	Bob

OrderItems		
OrderID	ProductID	Quantity
1001	P01	1
1001	P02	2
1002	P01	1
1002	P03	1

Products	
ProductID	ProductName
P01	Laptop
P02	Mouse
P03	Keyboard

Result

- Transitive dependency removed
- Each non-key attribute depends only on the key, nothing but the key

NORMALIZATION FUNDAMENTALS (2 OF 2)

Normalization proceeds through defined normal forms, each removing a specific kind of redundancy, until the design reaches a clean, related-table structure.

Form	Rule
1NF	Eliminate repeating groups; ensure every column holds a single, atomic (indivisible) value.
2NF	Must already be in 1NF, with no partial dependency of a non-key attribute on part of a composite key.
3NF	Must already be in 2NF, with no transitive dependency (a non-key attribute depending on another non-key attribute).
BCNF	A stronger version of 3NF: every determinant in the table must be a candidate key.

NORMALIZATION PROCESS



NORMALIZED DESIGN (FINAL RESULT)

Table	Key Columns	Depends On
CUSTOMER	PK CustomerID	—
ORDER	PK OrderID, FK CustomerID	CUSTOMER
ORDER_ITEM	PK (OrderID, ProductID)	ORDER, PRODUCT
PRODUCT	PK ProductID	—

KEY TAKEAWAY Always normalize up to 3NF for most business applications; use BCNF only when dealing with complex, unusual key relationships. Good normalization leads to good database design.

DESIGNING CUSTOMER DATABASES (1 OF 2)

A well-designed customer database helps organizations manage customer information efficiently, improve customer experience, and enable data-driven decisions.

KEY PRINCIPLES

- **Identify Requirements** — understand business needs and exactly what data must be stored.
- **Normalize Data** — reduce redundancy and enforce data integrity.
- **Define Relationships** — model entities and their relationships clearly.
- **Use Keys Effectively** — primary keys for uniqueness, foreign keys for integrity.
- **Ensure Security & Privacy** — protect sensitive customer data.

STEPS TO DESIGN A CUSTOMER DATABASE

#	Step	What You Do
1	Identify Requirements	Collect data needs: customers, orders, payments, etc.
2	Identify Entities	List main entities: Customer, Order, Product, Payment.
3	Define Attributes	Determine attributes for each entity.
4	Define Relationships	Identify how entities relate to each other.
5	Apply Keys & Constraints	Set primary keys, foreign keys, unique, not null, etc.
6	Normalize Data	Apply normalization, typically up to 3NF/BCNF.
7	Review & Optimize	Validate design, indexing, security, and performance.

WHY DESIGN IT RIGHT?

- Single source of truth for customer information.
- Eliminates redundancy and inconsistencies.
- Improves data quality, reporting, and analytics; supports growth.

KEY TAKEAWAY A well-designed customer database is the foundation for reliable operations, better customer insights, and long-term business success.

Designing Customer Databases

A structured approach to model, store, and manage customer information efficiently.

Key Principles



Understand Requirements

Identify what customer data needs to be captured and how it will be used.



Create an ER Model

Model entities, attributes, relationships, and keys.



Normalize Data

Organize data to reduce redundancy and avoid anomalies.



Enforce Integrity

Use primary keys, foreign keys, and constraints to ensure accurate data.



Optimize for Performance

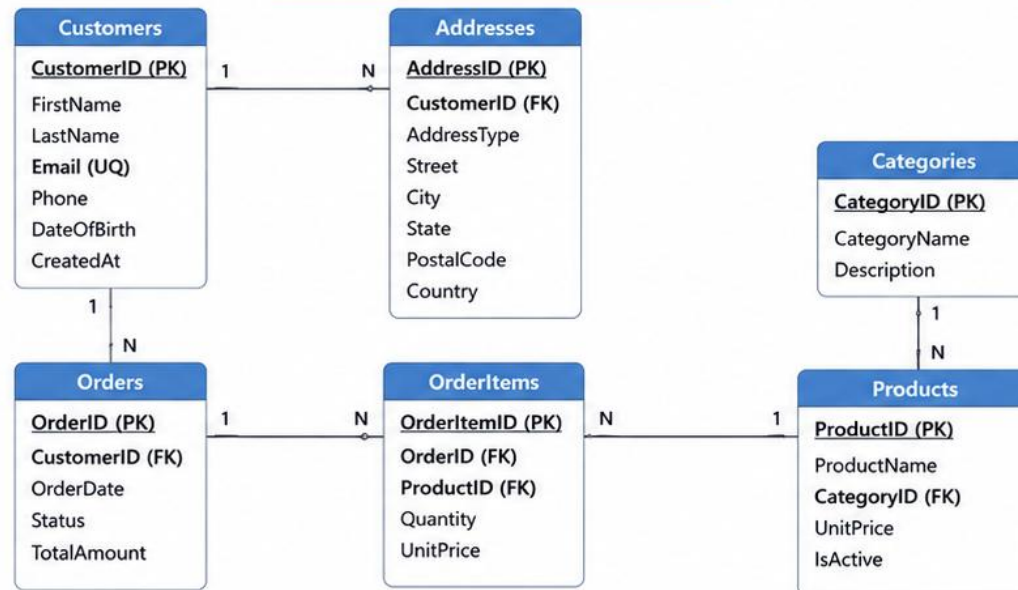
Index important columns and design for expected queries.



Plan for Growth

Design with scalability, flexibility, and maintenance in mind.

Entity Relationship Diagram



Sample Data Overview

Customers				Orders				OrderItems					Products			
CustomerID	FirstName	LastName	Email	OrderID	CustomerID	OrderDate	TotalAmount	OrderItemID	OrderID	ProductID	Quantity	UnitPrice	ProductID	ProductName	CategoryID	UnitPrice
1	Alice	Johnson	alice@example.com	101	1	2025-05-01	250.00	1	101	10	1	150.00	10	Laptop	1	150.00
2	Bob	Smith	bob@example.com	102	2	2025-05-02	125.50	2	101	20	2	50.00	20	Mouse	2	25.00
3	Carol	Williams	carol@example.com	103	1	2025-05-03	300.00	3	102	10	1	125.50	30	Keyboard	2	45.00

Benefits



Centralized and consistent customer information



Improved data integrity and accuracy



Better reporting and business insights



Efficient operations and scalability



Enhanced customer experience

Best Practices

- ✓ Use surrogate keys (e.g., ID)
- ✓ Apply appropriate data types and constraints
- ✓ Keep tables focused on a single subject
- ✓ Use meaningful relationships and cardinalities
- ✓ Document the design and rules



Oracle Database ER Diagram

Northstar Retail – Customer Order Management System

DIAGRAM LEGEND

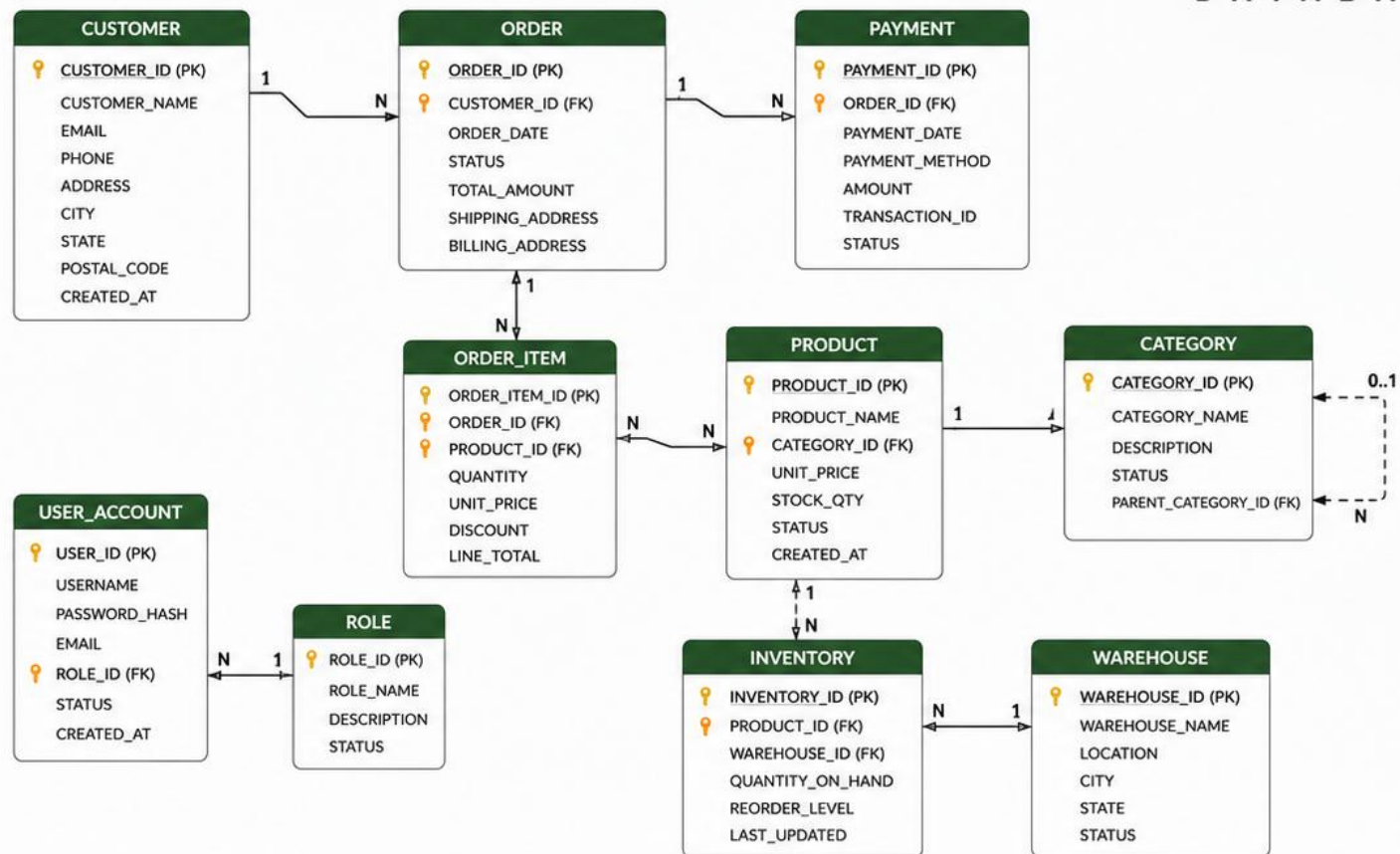
- Primary Key (PK)
- Foreign Key (FK)
- Relationship
- 1 One
- N Many
- - - Optional Relationship

DESIGN NOTES

- All primary keys are surrogate keys using NUMBER data type.
- Foreign key constraints ensure referential integrity.
- ON DELETE CASCADE is used for detail tables.
- Status columns control record lifecycle.

KEY BENEFITS

- ✓ Normalized design (3NF)
- ✓ Referential Integrity
- ✓ Scalable and Maintainable
- ✓ Supports Business Reporting

**RELATIONSHIP SUMMARY**

CUSTOMER 1	→ N ORDER	ORDER 1	→ N PAYMENT	PRODUCT 1	→ N INVENTORY
ORDER 1	→ N ORDER_ITEM	CATEGORY 1	→ N PRODUCT	WAREHOUSE 1	→ N INVENTORY
PRODUCT 1	→ N ORDER_ITEM	CATEGORY (parent) 1	→ N CATEGORY (child)	USER_ACCOUNT N	→ 1 ROLE

BUSINESS ENTITIES

DESIGNING CUSTOMER DATABASES (2 OF 2)

Turning the design into real tables, real queries, and a PostgreSQL-versus-Oracle implementation comparison.

TABLE DESIGN (SAMPLE COLUMNS)

Entity	Key Columns	Constraints
CUSTOMER	customer_id, first_name, last_name, email	PK, NOT NULL, UNIQUE(email)
ORDER	order_id, customer_id, order_date, total_amount	PK, FK -> CUSTOMER, NOT NULL
ORDER_ITEM	order_id, product_id, quantity, unit_price	PK (order_id, product_id), FK x2
PAYMENT	payment_id, order_id, payment_date, amount	PK, FK -> ORDER, NOT NULL

DATA INTEGRITY RECAP

- Primary Keys ensure each record is unique.
- Foreign Keys enforce referential integrity.
- UNIQUE prevents duplicates; NOT NULL ensures mandatory data.
- CHECK constraints enforce business rules.

Example Query (PostgreSQL)

```
SELECT c.customer_id, c.first_name,
       COUNT(o.order_id) AS total_orders,
       COALESCE(SUM(o.total_amount),0) AS spent
FROM customer c
LEFT JOIN "order" o
  ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.first_name;
```

POSTGRESQL VS ORACLE CONSIDERATIONS

PostgreSQL	Oracle
Uses SERIAL / IDENTITY for auto-increment.	Uses SEQUENCE + TRIGGER or IDENTITY.
Advanced features: JSONB, CTEs, window functions.	Enterprise features and partitioning at scale.
Strong referential integrity enforcement.	Robust security and auditing.

KEY TAKEAWAY A well-designed customer database is built once, on paper and in constraints, and then implemented consistently -- whether the target engine is PostgreSQL or Oracle.

TRY IT YOURSELF — EXERCISE 4: FILL DDL TODOS (STARTER)

Reinforces: PostgreSQL CREATE TABLE syntax for Northstar CRM -- fill in every blank before moving on.

notes/lab37-todos.sql (STARTER -- fill in every ____)

```
CREATE TABLE customer (  
  customer_id  VARCHAR(32) PRIMARY KEY,  
  full_name    ____ NOT NULL,  
  status       VARCHAR(16) NOT NULL,  
  created_at   TIMESTAMPTZ NOT NULL  
               DEFAULT ____,  
  CONSTRAINT customer_status_chk  
    CHECK (status IN (____))  
);  
  
CREATE TABLE account (  
  account_id  BIGINT GENERATED BY DEFAULT  
               AS IDENTITY PRIMARY KEY,  
  customer_id VARCHAR(32) NOT NULL  
               REFERENCES customer(____),  
  account_number VARCHAR(32) NOT NULL UNIQUE,  
  account_type VARCHAR(16) NOT NULL  
);  
  
-- TODO seed  
INSERT INTO customer (customer_id, full_name, status)  
VALUES  
  ('CUS-1001', '____', 'ACTIVE'),  
  ('CUS-1002', '____', 'ACTIVE');
```

#	What To Do
1	Create notes/lab37-todos.sql and paste the CREATE TABLE customer / account statements shown at left.
2	Fill every ____ blank: TEXT or VARCHAR(200) for full_name, now() for the default, 'ACTIVE','SUSPENDED' for the status list, customer_id for the FK reference, and Amina Khan / Ravi Singh for the two seed names.
3	Add the verify TODO comment: -- TODO Lab 37: run 04_verify.sql after apply (in lab).
4	Confirm no Oracle dialect crept in -- no NUMBER, no CASCADE CONSTRAINTS PURGE anywhere in your draft.

KEY TAKEAWAY TRY IT NOW -- This is a STARTER, not a solution: fill every ____ in exercises/exercise-04-fill-ddl-todos.md before moving on (workspace: examples/module-37-exercises).

TRY IT YOURSELF — EXERCISE 5: SEED AND VERIFY PLAN

Reinforces: seed ordering and verification queries for Amina/Ravi, planned before Lab 37 executes anything -- a documentation exercise.

STEPS (RECORD IN `notes/lab37-design.md`)

Step	What To Do
1 -- Seed Order	Insert customer rows before account rows -- an account's FK cannot reference a customer that doesn't exist yet.
2 -- Verify SQL	Write, on paper, the query you'll run after seeding: <code>SELECT customer_id, full_name FROM customer ORDER BY customer_id;</code>
3 -- Join Check	Sketch the join you'd run to list Amina's accounts by customer_id -- no execution yet.
4 -- No Execute	Do not run any of this against a live database in the pre-lab -- Lab 37 is where it actually executes.

Verify Queries (draft -- do not run yet)

```
-- Step 2: list customers
SELECT customer_id, full_name
FROM customer
ORDER BY customer_id;

-- Step 3: Amina's accounts
-- (paper join)
SELECT a.account_number,
       a.account_type
FROM account a
JOIN customer c
  ON a.customer_id = c.customer_id
WHERE c.customer_id = 'CUS-1001';
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 5 before continuing: `exercises/exercise-05-seed-and-verify-plan.md` (workspace: `examples/module-37-exercises`).

ENTERPRISE DATABASE DESIGN BEST PRACTICES (1 OF 2)

Well-designed databases are reliable, scalable, secure, and easy to maintain -- these best practices help build high-quality enterprise data solutions.



1. Business Requirements

Understand business processes and data needs. Identify entities, relationships, and rules. Design for current and future growth.



2. Model Data Correctly

Use appropriate data types. Normalize to eliminate redundancy (up to 3NF/BCNF). Use surrogate keys for simplicity.



3. Ensure Data Integrity

Define primary keys; use foreign keys. Apply NOT NULL, UNIQUE, CHECK, DEFAULT. Validate data at the database level.

4. DESIGN FOR PERFORMANCE



Index Smartly

Index primary keys, foreign keys, and frequently queried columns; avoid over-indexing.



Optimize Queries

Write efficient SQL, avoid SELECT *, use EXPLAIN to analyze plans.



Partition Large Tables

Improves performance, manageability, and availability.



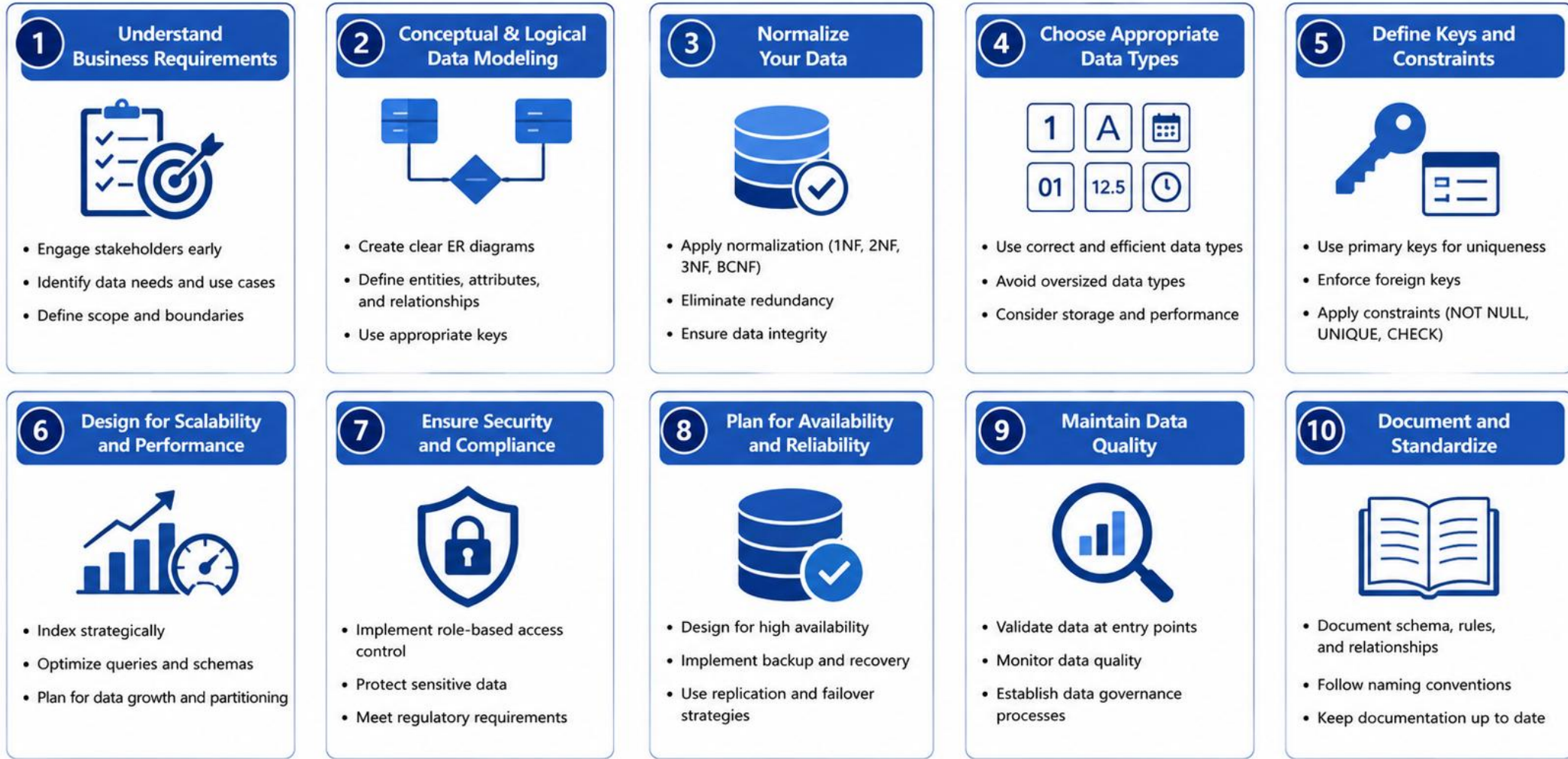
Use Views Wisely

Simplify complex queries and enhance security; avoid views in hot paths.

KEY TAKEAWAY Following best practices ensures data integrity, consistency, and accuracy; improves performance and scalability; and reduces development and maintenance costs.

Enterprise Database Design Best Practices

Guidelines for building scalable, secure, reliable, and maintainable database systems.



ENTERPRISE DATABASE DESIGN BEST PRACTICES (2 OF 2)

Scalability, security, maintainability, monitoring, and data governance round out a production-ready database design.



5. Scalability & Availability

Plan for growth in storage, users, and data volume. Use partitioning, replication, and clustering. Leverage read replicas; test backups regularly.



6. Security & Compliance

Enforce role-based access control (RBAC). Encrypt sensitive data at rest and in transit. Audit access; follow regulations (GDPR, PCI-DSS).



7. Maintainability & Standards

Follow consistent naming conventions. Document schema, relationships, and rules. Keep the design simple; review periodically.



8. Monitor & Optimize

Monitor performance, locks, and resource usage. Identify and fix slow queries. Rebuild indexes; update statistics regularly.



9. Data Governance

Define data ownership and stewardship. Maintain quality and consistent definitions. Establish lifecycle and retention policies.

GOOD VS POOR DESIGN

Poor (redundant): CustomerName repeated on every order row. Good (normalized): CUSTOMER 1:M ORDER -- no redundancy, consistent updates.

KEY TAKEAWAY A well-designed enterprise database is built on accuracy, integrity, performance, security, and maintainability -- follow best practices to deliver reliable data that drives business success.

TRY IT YOURSELF — EXERCISE 6: LAB 37 READINESS CHECKLIST

Capstone pre-lab check -- confirm artifacts, secrets hygiene, and next labs before opening Lab 37.

READINESS CHECKLIST (RECORD IN `notes/lab37-readiness.md`)

Check	What To Confirm
Deliverables Preview	List the four artifacts Lab 37 grades: schema SQL, seed SQL, an ER diagram, and design-decisions notes.
Env Reminder	Confirm database credentials stay in a local <code>.env</code> file only -- never committed to the repo.
Next Labs Preview	Note what comes after: Lab 38 tunes performance on this same schema; Lab 39 maps it with JPA entities.
Pass Mark	Confirm Exercise 4's DDL TODOs are fully filled in -- that is this checklist's pass condition.

Setup (PowerShell) -- from `EXERCISES-INDEX.md`

```
cd $env:USERPROFILE\java-bootcamp
New-Item -ItemType Directory -Force -Path examples\module-37-exercises | Out-Null
cd examples\module-37-exercises
java -version
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 6 before opening Lab 37: `exercises/exercise-06-lab37-readiness.md`.

LAB OVERVIEW -- POSTGRESQL DATABASE DESIGN

Lab 37: PostgreSQL Design for Customers and Accounts

BUSINESS SCENARIO

- The CRM stores customer identity, contact details, lifecycle status, postal addresses, and financial accounts -- React and Spring will persist to PostgreSQL, but tables come before ORM.
- Leadership freezes the gate: no CRM persistence merges without named constraints, exact money decimals, UTC timestamps, a least-privileged schema user, and seed fixtures CUS-1001 / CUS-1002.
- You design and implement the PostgreSQL CRM schema end to end: ER cardinalities, stable identifiers, a least-privileged CRM_APP user, and full DDL for four entities.

LEARNING OBJECTIVES

- Draw a normalized PostgreSQL CRM ER model with cardinalities.
- Create a least-privileged CRM_APP schema user (no DBA).
- Write CUSTOMER, ACCOUNT, ADDRESS, and status-history DDL with named constraints.
- Choose PostgreSQL money (NUMERIC(19,2)) and TIMESTAMPTZ types; index foreign keys.
- Seed and verify Amina/Ravi; prove constraints with negative tests and savepoints.

WHAT YOU BUILD

- lab37-crm with ER notes; PostgreSQL (Docker or shared instance) + CRM_APP least-privilege user; DDL for CUSTOMER / ACCOUNT / ADDRESS / CUSTOMER_STATUS_HISTORY with named PK/UK/FK/CK constraints and FK indexes; seed data for Amina (CUS-1001, ACTIVE, has an account) and Ravi (CUS-1002, PROSPECT, zero accounts); negative constraint tests with savepoints; drop/recreate proof.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE, has an account) and CUS-1002 (Ravi Singh, PROSPECT, zero accounts) anchor every example; correlation lab-request-001 appears in the seeded history row.

LAB TASKS AND DELIVERABLES (1 OF 2)

PostgreSQL Design for Customers and Accounts -- the eight implementation steps, in build order, plus a DDL excerpt.

IMPLEMENTATION STEPS (IN BUILD ORDER)

#	Implementation Step
1	Capture ER cardinalities (Customer 1 -- 0..* Account / Address / StatusHistory).
2	Choose stable identifiers: surrogate customer_id vs immutable public_id.
3	Connect to PostgreSQL (shared instance, or local Docker fallback).
4	Create the least-privileged CRM_APP schema user -- no DBA grants.
5	Create CUSTOMER, ACCOUNT, ADDRESS, CUSTOMER_STATUS_HISTORY with named constraints.
6	Add foreign-key and timeline indexes.
7	Seed Amina (CUS-1001, account) and Ravi (CUS-1002, no account).
8	Run negative constraint tests with SAVEPOINT; then drop and recreate cleanly.

CUSTOMER excerpt (PostgreSQL)

```
customer_id  BIGINT GENERATED BY DEFAULT AS IDENTITY,  
public_id    VARCHAR(36) NOT NULL,  
CONSTRAINT ck_customer_status CHECK (status IN ('PROSPECT','ACTIVE','SUSPENDED','CLOSED'))
```

KEY TAKEAWAY Steps 1-2 happen on paper before any SQL; step 4 (CRM_APP) is deliberately separate from schema creation; step 8's negative tests prove the constraints from steps 1-6 actually work.

LAB TASKS AND DELIVERABLES (2 OF 2)

PostgreSQL Design for Customers and Accounts -- expected deliverables and the 100-mark evaluation rubric.

EXPECTED DELIVERABLES

- ER notes/diagram with cardinalities and identifier rules.
- PostgreSQL runtime with persistent volume.
- CRM_APP least-privilege user script.
- Full DDL for all four tables + indexes.
- Seed script for Amina/Ravi + history correlation.
- Negative verification script with evidence; drop/recreate proof; design decisions + screenshots.

RUBRIC (100 MARKS)

Category	Marks
Environment	10
Core Implementation	30
Integration / Config Correctness	15
Failure Handling	15
Automated Verification	10
Security / Production Awareness	10
Documentation	10

Core Implementation is the largest single category. Integration/Config and Failure Handling are tied for second.

KEY TAKEAWAY FLOAT/BINARY money loses core marks; missing Ravi's zero-account case means incomplete seeds; DBA grants to CRM_APP is a security-practice honor violation.

MODULE 37 SUMMARY

In this module, you learned how to design well-structured, normalized databases in PostgreSQL, and how PostgreSQL compares with Oracle in enterprise systems.

KEY TOPICS COVERED

**Database Fundamentals & Architecture**

Relational concepts, and how PostgreSQL and Oracle are each built.

**ER Modeling & Relationships**

Entities, attributes, cardinality, and 1:1 / 1:M / M:M relationships.

**Keys & Constraints**

Primary keys, foreign keys, NOT NULL, UNIQUE, CHECK, referential integrity.

**Normalization**

Anomalies, 1NF through BCNF, and eliminating redundancy.

**Real-World Design**

Designing a normalized customer database from requirements to DDL.

**Enterprise Best Practices**

Performance, scalability, security, maintainability, and governance.

POSTGRESQL VS ORACLE -- KEY POINTS

PostgreSQL	Oracle
Open source, community-driven, cost-effective; SERIAL/IDENTITY; strong JSON/CTE/window-function support.	Enterprise-grade, feature-rich; SEQUENCE+TRIGGER/IDENTITY; RAC, Data Guard, advanced security and auditing.

KEY TAKEAWAY You can now design reliable, scalable, and maintainable databases in PostgreSQL that ensure data integrity, support business operations, and deliver high performance -- and explain how the same design transfers to Oracle.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of database design goals, ER diagrams, normal forms, and uniqueness constraints.

1. Which of the following is the primary goal of database design?

- A. To make the database as large as possible
- B. To store as much data as possible without constraints
- C. To organize data efficiently, ensuring integrity**
- D. To avoid using relationships between tables

3. Which normal form removes partial dependencies on a subset of a primary key?

- A. 1NF
- B. 2NF**
- C. 3NF
- D. BCNF

2. Which diagram is used to represent entities, attributes, and relationships?

- A. Data Flow Diagram (DFD)
- B. Entity Relationship Diagram (ERD)**
- C. State Transition Diagram
- D. Sequence Diagram

4. Which constraint ensures that values in a column are unique across the table?

- A. PRIMARY KEY
- B. FOREIGN KEY
- C. UNIQUE**
- D. CHECK

KEY TAKEAWAY Good database design organizes data efficiently while ensuring integrity and performance; ERDs model entities/attributes/relationships; 2NF removes partial dependencies; UNIQUE enforces distinct values.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on foreign keys, PostgreSQL primary key syntax, filtering rows, and the purpose of normalization.

5. Which command defines a primary key in PostgreSQL?

- A. CREATE PRIMARY KEY
- B. ALTER TABLE ... ADD PRIMARY KEY**
- C. SET PRIMARY KEY
- D. PRIMARY KEY CREATE

7. Which SQL clause is used to filter rows based on a condition?

- A. ORDER BY
- B. HAVING
- C. WHERE**
- D. GROUP BY

6. Which of the following best describes a foreign key?

- A. A key that uniquely identifies a row in the same table
- B. References another table's primary key**
- C. A key used to encrypt data
- D. A key that is automatically generated by the database

8. What is the purpose of normalization?

- A. To speed up data entry
- B. To reduce data redundancy and improve data integrity**
- C. To increase the number of tables
- D. To make queries more complex

KEY TAKEAWAY ALTER TABLE ... ADD PRIMARY KEY defines a primary key after the fact; a foreign key enforces referential integrity to another table; WHERE filters rows; normalization exists to reduce redundancy and improve integrity.

MODULE 37 COMPLETE

PostgreSQL Database Design

You can now design robust, normalized databases in PostgreSQL -- keys, constraints, relationships, and best practices -- and explain how that design transfers to Oracle.